

Experiment 6

Title: Comparison of Docker Run and Docker Compose

PART A - THEORY

1. Objective To understand the relationship between docker run and Docker Compose, and to compare their configuration syntax and use cases.

2. Background Theory

2.1 Docker Run (Imperative Approach) The docker run command is used to create and start a container from an image. It requires explicit flags for:

- Port mapping (-p)
- Volume mounting (-v)
- Environment variables (-e)
- Network configuration (--network)
- Restart policies (--restart)
- Resource limits (--memory, --cpus)
- Container name (--name)

This approach is imperative, meaning you specify step-by-step instructions.

Example:

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v ./html:/usr/share/nginx/html \
  -e NGINX_HOST=localhost \
  --restart unless-stopped \
  nginx:alpine
```

2.2 Docker Compose (Declarative Approach) Docker Compose uses a YAML file (docker-compose.yml) to define services, networks, and volumes in a structured format. Instead of multiple docker run commands, a single command is used: docker compose up -d. Compose is declarative, meaning you define the desired state of the application.

Equivalent Compose file:

```
version: '3.8'

services:
  nginx:
```

image: nginx:alpine

container_name: my-nginx

ports:

- "8080:80"

volumes:

- ./html:/usr/share/nginx/html

environment:

NGINX_HOST: localhost

restart: unless-stopped

3. Mapping: Docker Run vs Docker Compose

- -p 8080:80 equivalent to ports:
- -v host:container equivalent to volumes:
- -e KEY=value equivalent to environment:
- --name equivalent to container_name:
- --network equivalent to networks:
- --restart equivalent to restart:
- --memory equivalent to deploy.resources.limits.memory
- --cpus equivalent to deploy.resources.limits.cpus

4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling (Example: docker compose up --scale web=3)

PART B - PRACTICAL TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run Execute:

```
docker run --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
4524adc22ad136ff5292e6d25cf1f713d4d64a1b59d47e0c7f5aa69ca35042c4
```

Verify: docker ps

```
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
4524adc22ad1   nginx:alpine "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   lab-nginx
```

Access: http://localhost:8081

⏪ ⏩ ↻ http://localhost:8081

ðŸ”Œ Nginx is working

Step 2: Run Same Setup Using Docker Compose Create docker-compose.yml:

```
GNU nano 7.2
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Task 2: Multi-Container Application Objective: Deploy WordPress with MySQL using:

1. Docker Run (manual way)
2. Docker Compose (structured way)

A. Using Docker Run

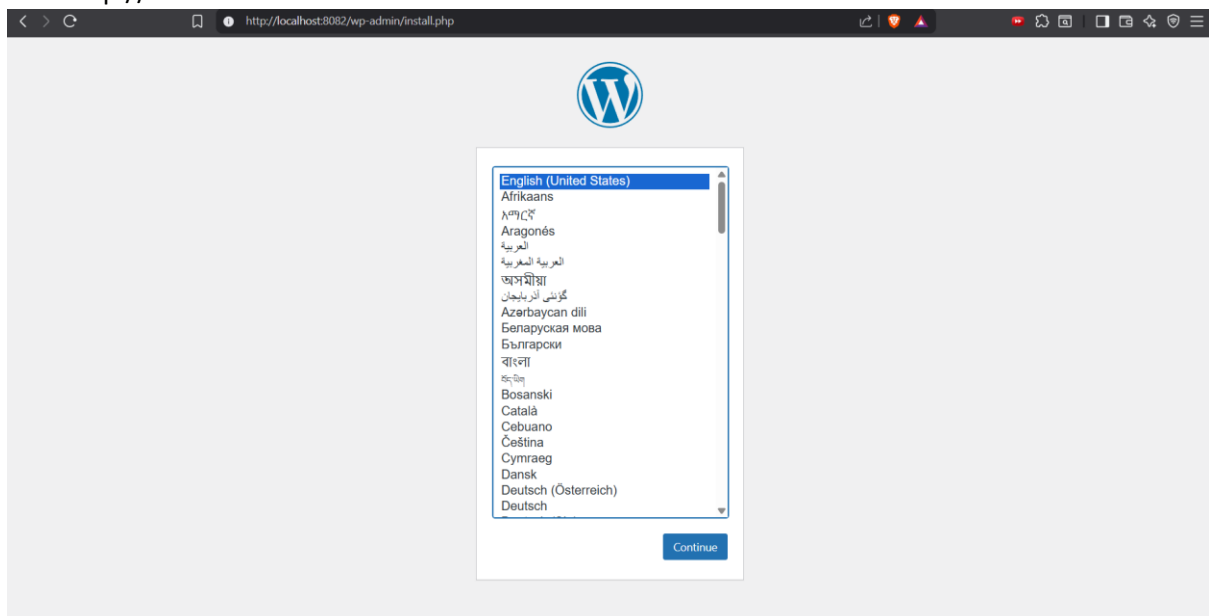
1. Create network: docker network create wp-net
2. Run MySQL:

```
--name mysql \
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
cdb381ffdb7b8d8b14832d7bfde88e28c1f10e2b63531464f30566f7fb1e1ab8
```

3. Run WordPress:

```
--name wordpress \
--network wp-net \
-p 8082:80 \
-e WORDPRESS_DB_HOST=mysql \
-e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
477bc7b9a25c4a9746e3fdc212e36a720d62916c863d501b835001317ec7b0fa
```

Test: <http://localhost:8082>



B. Using Docker Compose Create docker-compose.yml:

```
GNU nano 7.2
services:
  webapp:
    image: node.18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

Stop: docker compose down -v

```
WARN[0000] /home/anirudh_chawla/exp6/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 4/4
✓Container exp6-wordpress-1 Removed 1.4s
✓Container exp6-mysql-1 Removed 1.5s
✓Volume exp6-mysql_data Removed 0.1s
✓Network exp6_default Removed 0.3s
```

PART C - CONVERSION & BUILD-BASED TASKS

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application Given Docker Run Command:

```
docker run -d \
--name webapp \
-p 5000:5000 \
-e APP_ENV=production \
-e DEBUG=false \
--restart unless-stopped \
node:18-alpine
```

Student Task:

1. Write an equivalent docker-compose.yml

```
GNU nano 7.2
services:
  webapp:
    image: node.18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

2. Ensure: Same container name, same port mapping, same environment variables, same restart policy.
3. Run using: docker compose up -d

```
NAME                IMAGE                COMMAND                SERVICE    CREATED
STATUS
webapp              node:18-alpine      "docker-entrypoint.s..."  webapp     About a minute ago
Restarting (0) 38 seconds ago
```

4. Verify using: docker compose ps

Problem 2: Volume + Network Configuration Given Docker Run Commands:

```
docker network create app-net
```

```
docker run -d \
```

```
--name postgres-db \
```

```
--network app-net \
```

```
-e POSTGRES_USER=admin \
```

```
-e POSTGRES_PASSWORD=secret \
```

```
-v pgdata:/var/lib/postgresql/data \
```

```
postgres:15
```

```
docker run -d \
```

```
--name backend \
```

```
--network app-net \
```

```
-p 8000:8000 \
```

```
-e DB_HOST=postgres-db \
```

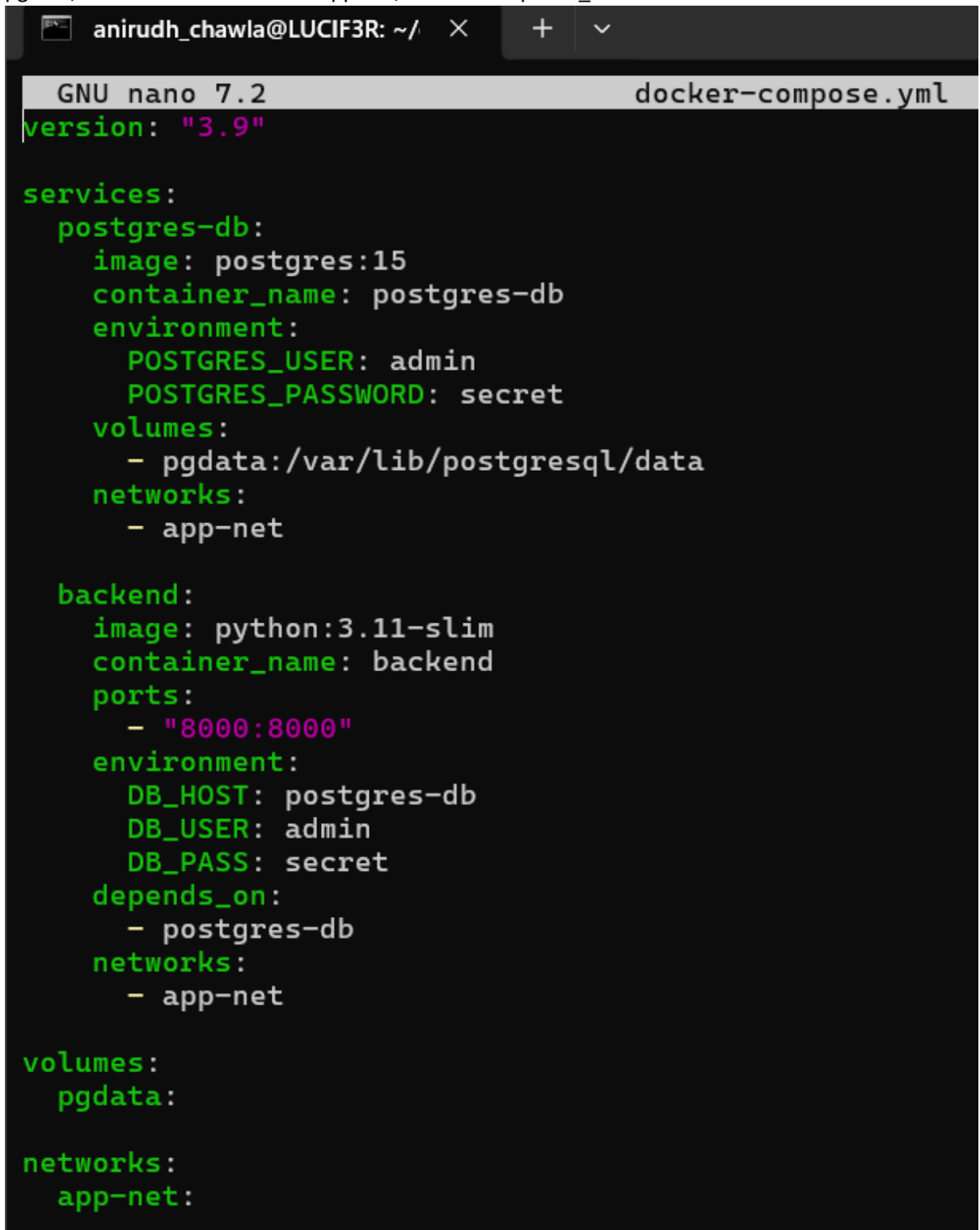
```
-e DB_USER=admin \
```

```
-e DB_PASS=secret \
```

```
python:3.11-slim
```

Student Task:

1. Create a single docker-compose.yml file that defines both services, creates named volume pgdata, creates custom network app-net, and uses depends_on.



```
anirudh_chawla@LUCIF3R: ~/ × + ∨  
GNU nano 7.2 docker-compose.yml  
version: "3.9"  
  
services:  
  postgres-db:  
    image: postgres:15  
    container_name: postgres-db  
    environment:  
      POSTGRES_USER: admin  
      POSTGRES_PASSWORD: secret  
    volumes:  
      - pgdata:/var/lib/postgresql/data  
    networks:  
      - app-net  
  
  backend:  
    image: python:3.11-slim  
    container_name: backend  
    ports:  
      - "8000:8000"  
    environment:  
      DB_HOST: postgres-db  
      DB_USER: admin  
      DB_PASS: secret  
    depends_on:  
      - postgres-db  
    networks:  
      - app-net  
  
volumes:  
  pgdata:  
  
networks:  
  app-net:
```

2. Bring up services using one command.

```

obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 6/7
✓Image python:3.11-slim Pulled 8.4s
  Network exp6_app-net Creating 0.0s
[+] up 10/10ound orphan containers ([webapp]) for this project. If you removed or r
✓Image python:3.11-slim Pulled 8.4s
✓Network exp6_app-net Created 0.0s
✓Volume exp6_pgdata Created 0.0s
✓Container postgres-db Started 0.3s
✓Container backend Started 0.3s

```

```

obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 5/5
✓Container backend Removed 0.0s
✓Container postgres-db Removed 0.2s
✓Container webapp Removed 0.0s
✓Volume exp6_pgdata Removed 0.0s
✓Network exp6_app-net Removed 0.3s

```

3. Stop and remove everything properly.

Task 4: Resource Limits Conversion Given Docker Run Command:

```

docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine

```

Student Task:

1. Convert this to Docker Compose.
2. Add resource limits using: `deploy > resources > limits`.

3.

```
GNU nano 7.2
version: '3.8'
services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 256M
```

PART D - USING DOCKERFILE INSTEAD OF STANDARD IMAGE

Task 5: Replace Standard Image with Dockerfile (Node App) Instead of using prebuilt images like image: node:18-alpine, students must create their own Dockerfile, build image using Compose, and run container from that build.

Scenario: You are given docker run -d -p 3000:3000 node:18-alpine. Now you must:

1. Create a simple Node.js app
2. Write a Dockerfile
3. Use Docker Compose build: option

Step 1: Create app.js

```
GNU nano 7.2 app.js *
const http = require('http');
http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);|
```

Step 2: Create Dockerfile

```

GNU nano 7.2 Dockerfile *
FROM node:18-alpine
WORKDIR /app
COPY app.js
EXPOSE 3000
CMD ["node", "app.js"]

```

Step 3: Create docker-compose.yml

```

GNU nano 7.2 docker-compose.yml *
version: '3.8'
services:
  nodeapp:
    build:
      context:
        dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"

```

Student Task:

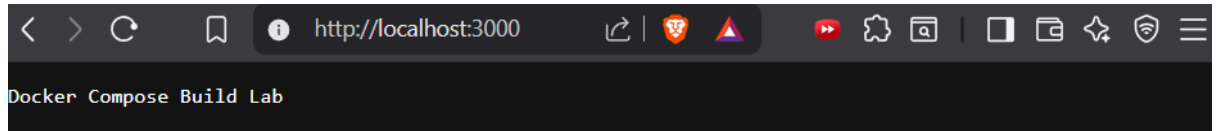
1. Build and run using: `docker compose up --build -d`

```

WARN[0000] /home/anirudh_chawla/exp6/docker-compose.yml: the attribute 'version' is
obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.3s (10/10) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 500B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 119B 0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 153B 0.0s
=> [2/3] WORKDIR /app 0.0s
=> [3/3] COPY app.js . 0.0s
=> exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:93f75belad4c7ffa0fc726289f693f58118c64ab062 0.0s
=> => exporting config sha256:12ab4374da5d73131af5014cb04a563a3052e2edc22e9 0.0s
=> => exporting attestation manifest sha256:c69b7c5d0ea664ee5204378ac787822 0.0s
=> => exporting manifest list sha256:7fd0f369196c32d6ff7be7317c88be723d7ca 0.0s
=> => naming to docker.io/library/exp6-nodeapp:latest 0.0s
=> => unpacking to docker.io/library/exp6-nodeapp:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 2/2
✔Image exp6-nodeapp Built 0.4s
✔Container custom-node-app Started 0.2s

```

3. Verify in browser: <http://localhost:3000>



4. Modify message in app.js, rebuild and observe changes.
5. Explain difference between image: and build:.
Ans: image: is the specification of which image to use, whereas build: is used to tell which Dockerfile to use to build the image that will be used to run the container.

Task 6: Multi-Stage Dockerfile with Compose Requirement: Create a simple Python FastAPI or Node production-ready app using:

- Multi-stage Dockerfile
- Smaller final image
- Use Compose to build it

Must:

1. Write multi-stage Dockerfile

```
GNU nano 7.2 Dockerfile
FROM node:18-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app .
EXPOSE 8080
CMD ["npm", "start"]
```

2. Use build: in Compose

```
GNU nano 7.2 docker-compose.yml *
version: '3.8'
services:
  app:
    build:
      context: .
    ports:
      - "8080:8080"
    environment:
      - NODE_ENV=production
    volumes:
      - ./app|
```

3. Add environment variables
4. Add volume mount for development mode
5. Compare image size: docker images

Experiment 6 B

Multi-Container Application using Docker Compose (WordPress + Database)

1. Objective To deploy a multi-container application using Docker Compose, consisting of:

- WordPress (frontend + PHP)
- MySQL database (backend)

Also: Understand container networking & volumes, learn how to scale services, compare with Docker Swarm for production deployment.

2. Prerequisites

- Docker installed
- Docker Compose (comes with modern Docker)
- Basic understanding of containers

3. Architecture Overview User (Browser) -> WordPress Container -> MySQL Container -> Persistent Volume (Database Storage)

- WordPress connects to MySQL using service name (DNS inside Docker network)
- Data is persisted using volumes

Steps

Step 1: Create Project Directory

```
mkdir wp-compose-lab
```

cd wp-compose-lab

Step 2: Create docker-compose.yml

```
GNU nano 7.2                                     docker-compose.yml
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Explanation of Key Sections

- **services:** Defines containers (db -> MySQL database, wordpress -> application)
- **depends_on:** Ensures DB starts before WordPress

- **environment:** Used to configure DB credentials and connection
- **volumes:** Persist data even if containers are deleted
- **ports:** Exposes WordPress on <http://localhost:8080>

Step 3: Start Application docker-compose up -d

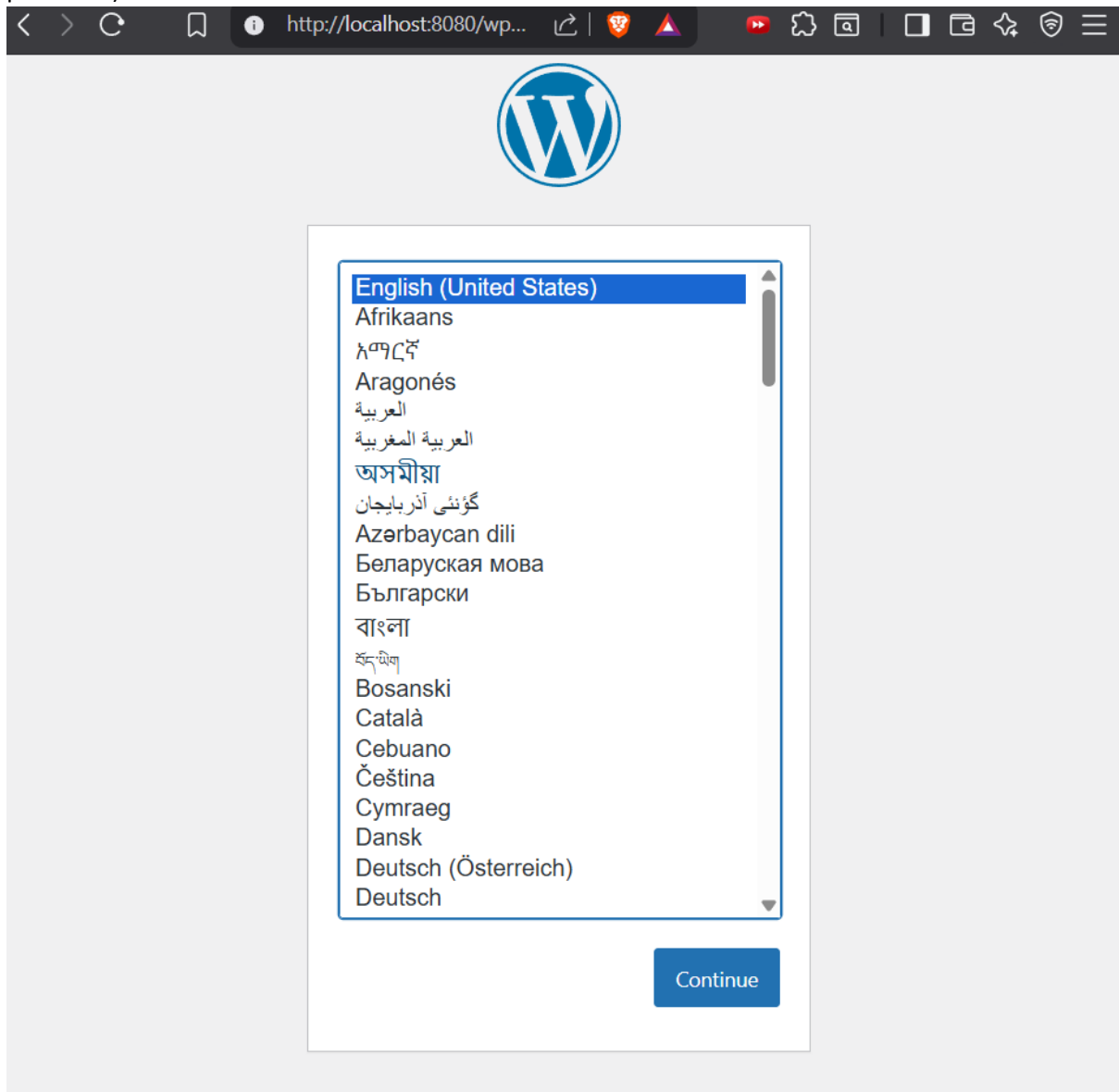
```
te 'version' is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 5/5
✓Network wp-compose-lab_default Created 0.0s
✓Volume wp-compose-lab_db_data Created 0.0s
✓Volume wp-compose-lab_wp_data Created 0.0s
✓Container wordpress_db Started 0.2s
✓Container wordpress_app Started 0.3s
```

What happens: Images are pulled, Network is created, Containers are started, DNS-based service discovery enabled.

Step 4: Verify Containers docker ps (Expected: wordpress_app, wordpress_db)

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3424097ab9421	wordpress:latest	"docker-entrypoint.s..."	7 seconds ago	Up 7 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	wordpress_app
a17a29199d4d	mysql:5.7	"docker-entrypoint.s..."	7 seconds ago	Up 7 seconds	3306/tcp, 33060/tcp	wordpress_db
36c59d832bdb	exp6-nodeapp	"docker-entrypoint.s..."	17 minutes ago	Up 17 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	custom-node-app
3e01313ac91	wordpress:latest	"docker-entrypoint.s..."	About an hour ago	Up About an hour	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp	wordpress
55e15ecaf9f0	mysql:5.7	"docker-entrypoint.s..."	About an hour ago	Up About an hour	3306/tcp, 33060/tcp	mysql

Step 5: Access WordPress Open browser: <http://localhost:8080> (Complete setup, enter title, user, password)



Step 6: Check Volumes docker volume ls (db_data -> database persistence, wp_data -> WordPress files)

```
DRIVER      VOLUME NAME
local       5b25b03fcbd30f072a3c2841dda282dbc03630272a856fb95892789634a95915
local       8e22d80ef6accdb472c02d4a8b7c977ec2d374b0a59c165d5ea0815832f25275
local       9a116c9437d1d0b1ec608b91978e21ae2d64cc0a1c8c94c2a993e2dbdbcd89da
local       48fdefa035bc8abd516b974b005e8c2da1eec7fd0b9372555a764f8bacfd27d7
local       441ed787faa660c56d13c08ea1becaa8729a414d1c409eff372dd7baf18d6e62
local       app-volume
local       cb922b95c451c4a32b282b95e1eb8b5efb6337e49bb9cfc333abc9f431262cf7
local       e1f81d6fc8ca4a1e484eb6bbe4064cc5320e9fe4d3a1c0d865628a808ec3cdad
local       eeee3012b86a1237a291321574d2a1bf1cafbcb9eb9f44f552ec6bb63c2d946b9
local       fceffbc8008555e00094209b8fcd72118ea38a7d94f8b59aff82f4fc4bde0f57
local       mydata
local       mysql-data
local       pgdata
local       redis-data
local       wp-compose-lab_db_data
local       wp-compose-lab_wp_data
```

Step 7: Stop Application docker-compose down (Containers removed, Volumes remain intact)

```
te 'version' is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] down 3/3
✓Container wordpress_app      Removed      1.3s
✓Container wordpress_db      Removed      1.7s
✓Network wp-compose-lab_default Removed      0.2s
```

6. **Scaling in Docker Compose** Method 1: Scale WordPress Containers docker-compose up --scale wordpress=3

```
te 'version' is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 2/2
✓Network wp-compose-lab_default Created      0.0s
✓Container wordpress_db      Created      0.0s
WARNING: The "wordpress" service is using the custom container name "wordpress_app"
. Docker requires each container to have a unique name. Remove the custom name to s
cale the service
```

Result: 3 WordPress containers running. Problem: All try to use same port (8080), no load balancing. Solution: Use Reverse Proxy (Nginx). Add another service, then configure load balancing manually.

```
GNU nano 7.2                                docker-compose.yml *
```

```
version: '3.9'

services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:8080"

  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
```

Limitations of Compose Scaling:

- No built-in load balancing
- No auto-healing
- Single host only

- Not production-ready for scaling

7. Running Same Setup with Docker Swarm Step 1: Initialize Swarm: docker swarm init

```
2.22.87.136
Swarm initialized: current node (ify2vksfmuclt85lars33nqa5) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-165kee8g9o2a50elsje4o1uh6gmpnan3ddg5mfikvzt5
    tenzus-35q154kjdtalvxnor01vs8ij8 172.22.87.136:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow th
e instructions
```

Step 2: Deploy Stack: docker stack deploy -c docker-compose.yml wpstack

```
.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
Creating service wpstack_nginx
```

Step 3: Scale Service: docker service scale wpstack_wordpress=3

```
s=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

What Changes in Swarm?

Feature	Docker Compose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Manual	Built-in
Load balancing	No	Yes (internal LB)
Self-healing	No	Yes
Rolling updates	No	Yes
Networking	Basic	Overlay network

7. Benefits of Docker Swarm

- Built-in load balancing

- Automatic container restart (self-healing)
- Horizontal scaling across nodes
- Rolling updates without downtime
- Service abstraction (not individual containers)

8. Challenges / Limitations of Swarm

- Less popular than Kubernetes
- Limited ecosystem
- Less flexible scheduling
- Fewer enterprise features

9. Key Learning Outcomes

- Multi-container apps require orchestration
- Docker Compose is ideal for: Development, Testing, Learning
- Docker Swarm is useful for: Simple production clusters, Easy scaling without Kubernetes complexity

10. Conclusion This experiment demonstrated:

- How to deploy WordPress + MySQL using Docker Compose
- How containers communicate using internal networking
- Importance of volumes for persistence
- Scaling limitations of Compose
- Advantages of using Docker Swarm for production-ready deployments